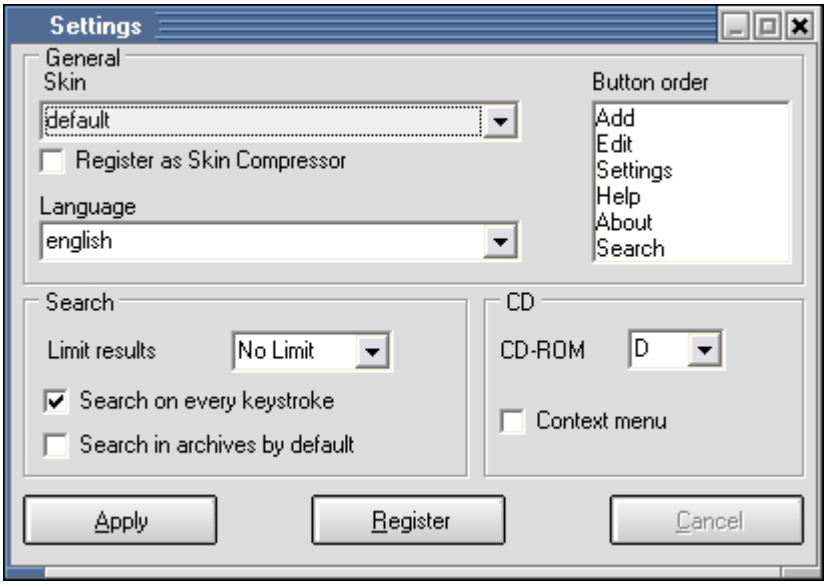
Programa:	BG CD Manager 1.41
Protección:	UPX + Trial 7 días
Objetivos:	Desempakar y evitar la limitación temporal
URL descarga:	me lo podéis pedir
Dificultad:	Más fácil no la encontraréis ;-)
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6 + LordPE
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 01-01-06

Hola a todo CracksLatinoS..!!

Bueno a ver si este es el despertar pues llevo tiempo sin escribir ningún tute, varias son las razones de que no lo haya hecho hasta entonces, la falta de tiempo, mi HD que tengo estropeado, snif, snif, y demás situaciones que me empujaron a dejar a un ladito el cracking, eso sí mirando siempre de reojo, jeje.. sigo al día en la lista eso no me lo quita nadie es uno de los mejores alicientes y no pienso sacrificar, en fin que aquí vuelvo a la carga con una aplicación más, he de reconocer que no sé que saldrá de aquí pues no tengo el parche ni nada, acabo de instalar la app y comienzo a mirarla a la par que escribo el tute, los que me conocen saben que es la manera que yo prefiero a la hora de "tutear" jeje.. o sea de escribir tutes ;-)

Al Atakeeeeeeeeee...

Nsda más instalar la aplicación si la ejecutamos recibimos una pantallita como esta:



I
En esta pantalla se definen los parámetros de configuración, sólo aparecerá una vez, a la siguiente cargará directamente el programa y no la volveremos a ver, es por esto que debemos configurar todo en cuanto aparezca, pero ¿cómo sabe el programa que es la primera vez y no la cuarta? bien sencillo, en el S.O. Window$ existen una serie de archivos encargados de almacenar la configuración de los programas instalados así como los del propio sistema operativo, estos archivos son accesibles mediante el "regedit.exe" o rgistro de sistema, pues bien allí guarda unos valores donde los revisa en el inicio del programa, si encuentra cierta información supondrá que ya lo hemos ejecutado antes y configurado y si no pues nos muestra esta pantalla para que lo hagamos ahora. Unas de las apis más usadas para acceder al registro son:

RegQueryValueA
RegQueryValueExA

Y las mismas pero para almacenar serían:

RegSetValueA
RegSetValueExA

Esto de momento no es importante pero no sé he visto interesante explicar un poquitín cómo funciona el sistema.

Si nos fijamos ya en la misa pantalla de configuración nos aparece la opción de registro accesible mediante el botón "Register" si pulsamos sobre él se desplegará el cuadro de registro donde nos solicita un código de licencia (License Key) como no lo sabemos y queremos forzar un primer ataque escribiremos uno cualquiera como este ;-)

El famoso 121212, pobre del programador que lo utilice alguna vez como serial válido se va a registrar medio planeta sin pagar ni un duro, jeje.. no es el caso pues ni si quiera se activa el botón "Save" desde el que comprobaremos la validez del serial, para que este se active deberemos poner un serial de al menos 30 dígitos:

ahora sí está activado el botón, si procedemos a pulsar en "Save" el programa comprobará el srial y mirad que mensajito nos tenía guardado para los seriales no_válidos ;-)

No parece el típico MessageBox pero bueno intentaremos hallar la api responsable. Hasta ahora hemos dado un primer vistazo al programa y hemos ido viendo lo que hacía, etc, etc., pero esto no es suficiente, deberemos lanzar un primer ataque.

En efecto chabales y a eso vamos, todavía no hemos mirado si el ejecutable está empakado y llegó el momento de hacerlo, rescatamos PEiD y vemos...

UPX 0.89.6 - 1.02 / 1.05 - 1.24 (Delphi) stub -> Markus & Laszlo

Vaya un UPX! este casi se desempaka sólo, jeje.. un respeto que yo sería incapaz de programar algo similar, lo que sí tenemos son muchas ganas de crackear y esto lo va a sufrir el pobre UPX, además no hará mucho enviaron a la lista un Olly 1.10 con todos o casi todos los plugins necesarios para el combate, no los enumeraré todos pero son bastantes, con él trabajaremos, creedme que nos ahorrará bastante tiempo ;-)

Pues venga abrimos el Olly_1.10_full y nos paramos en el EntryPoint del programa, es decir el punto de entrada cuando ya está empakado, inicialmente este punto era otro pero al comprimirlo se desvió hacia el inicio del packer (UPX en este caso) uno de los primeros objetivos a la hora de desempakar algo es hallar el punto de entrada REAL de la aplicación, es decir, el punto de entrada inicial "antes de empakarlo" esto es importante porque si conseguimos un archivo descomprimido "dumpeado" necesitaremos saber este dato para que arranque desde allí sino no funcionará ;-(

Buscando el OEP...

En fin pocas vueltas a este punto pues UPX no utiliza Stolen Bytes, ni ofuscación ni nada de eso, ya antes os comenté la fragilidad de este packer, lo único que debemos hacer es lo siguiente:

1.- Abrimos un Olly y cargamos "BGCDManager.exe"

```
Address   Hex dump        Disassembly                          Comment
005010A0  60              PUSHAD                               EP==5010A0
005010A1  BE 00B04A00     MOV ESI,BGCDMana.004AB000
005010A6  8DBE 0060F5FF   LEA EDI,DWORD PTR DS:[ESI+FFF56000]
005010AC  C787 D0940D00   MOV DWORD PTR DS:[EDI+D94D0],2B20B9F8
005010B6  57              PUSH EDI
005010B7  83CD FF         OR EBP,FFFFFFFF
005010BA  EB 0E           JMP SHORT BGCDMana.005010CA
005010BC  90              NOP
005010BD  90              NOP
```

Ese es el EP (que no el OEP) del programa, a desde aquí hacemos un scroll hacia abajo con la rueda del ratón hata encontrar el salto al OEP, es fácil de reconocer ya que salta fuera de la sección actual (4AB000-502000) y es este:

```
Address   Hex dump        Disassembly                          Comment
005011F9  09C0            OR EAX,EAX
005011FB  74 07           JE SHORT BGCDMana.00501204
005011FD  8903            MOV DWORD PTR DS:[EBX],EAX
005011FF  83C3 04         ADD EBX,4
00501202  EB E1           JMP SHORT BGCDMana.005011E5
00501204  FF96 804D1000   CALL DWORD PTR DS:[ESI+104D80]
0050120A  61              POPAD
0050120B  E9 64FFFCFF     JMP BGCDMana.004D1174                ;salto al OEP!
00501210  2812            SUB BYTE PTR DS:[EDX],DL
00501212  50              PUSH EAX
00501213  0038            ADD BYTE PTR DS:[EAX],BH
00501215  1250 00         ADC DL,BYTE PTR DS:[EAX]
00501218  D0A44D 0000000  SHL BYTE PTR SS:[EBP+ECX*2],1
0050121F  0000            ADD BYTE PTR DS:[EAX],AL
00501221  0000            ADD BYTE PTR DS:[EAX],AL
```

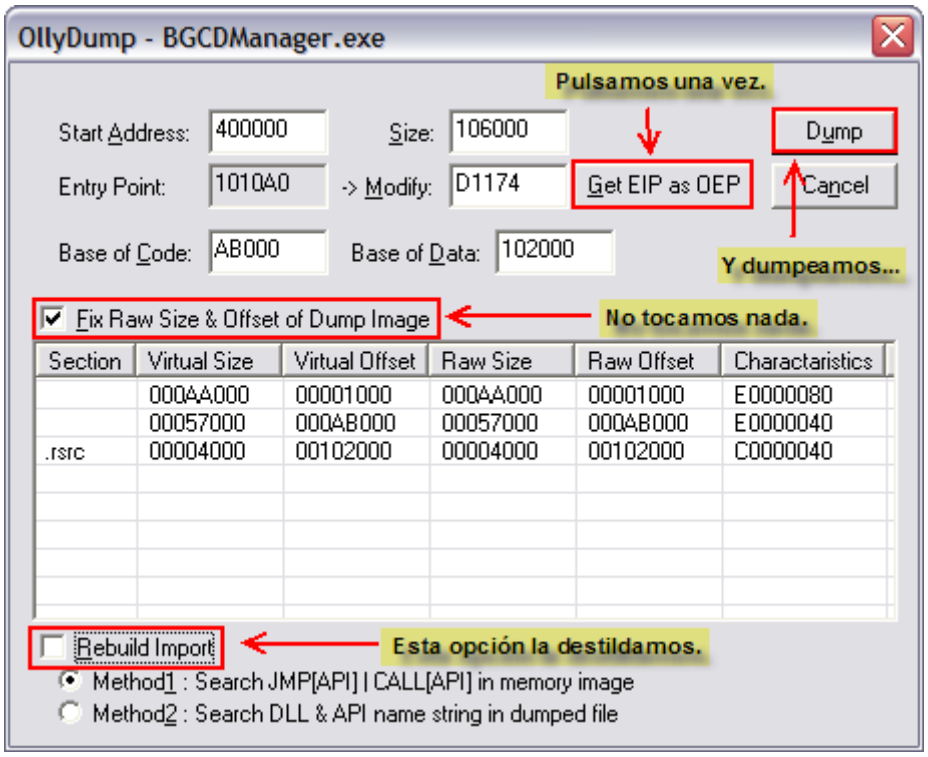
Bueno pues ya lo tenemos, así de fácil ya os lo comenté, el siguiente paso será llegar hasta él, ponemos un BP en el salto y pulsamos F9, al detenernos allí pulsamos F7/F8 y superamos la instrucción, ya está nos plantamos en el OEP ;-)

```
Address   Hex dump        Disassembly                          Comment
004D1174  55              PUSH EBP
004D1175  8BEC            MOV EBP,ESP
004D1177  B9 17000000     MOV ECX,17                           OEP==4D1174!!
004D117C  6A 00           PUSH 0
004D117E  6A 00           PUSH 0
004D1180  49              DEC ECX
004D1181  75 F9           JNZ SHORT BGCDMana.004D117C
004D1183  53              PUSH EBX
004D1184  56              PUSH ESI
004D1185  57              PUSH EDI
004D1186  B8 540E4D00     MOV EAX,BGCDMana.004D0E54
004D118B  E8 5057F3FF     CALL BGCDMana.004068E0
004D1190  8B1D 98984D00   MOV EBX,DWORD PTR DS:[4D9898]        BGCDMana.004DA7D4
```

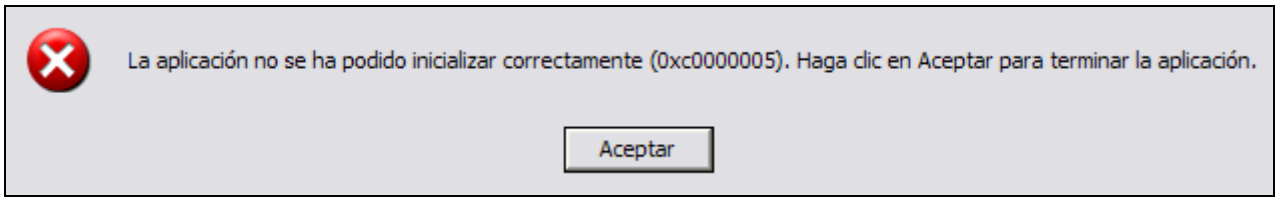
Lo anotamos que después hará falta para que el dumpeado funcione, recordad que este era el punto de entrada (EntryPoint) del programa antes de comprimirlo con UPX, por eso ahora se le llama OEP (Original Entry Point) que es el punto de entrada "real" recordad que el Entry Point es el inicio del packer y lo teníamos en 5010A0 donde había un PUSHAD (Push All Purpose-Registers en 32 Bits) para más detalle ver el magnífico curso de Ricardo "Introducción al cracking con Ollydbg" donde matiza todos estos elementos dando una explicación detallada de cada uno de ellos y sus aplicaciones, en lo que a nosotros respecta nos quedamos en el OEP del programa, es un UPX y no tiene dificultad, no ocurre lo mismo con packers como ASProtect 2.1 SKE, ACProtect, UltraProtect, etc.. donde los Stolen bytes hacen su aparición siendo a veces casi imposible deducir cuales son, pues lo dicho anotamos el dato:

OEP: 4D1174

Y seguimos... el proximo paso será recuperar la IAT, aunque en este packer tampoco es necesario y según con qué opción se dumpee se puede evitar el uso del ImpRec (Import Reconstructor 1.6 la versión que yo utilizo) pero antes de esto vamos a obtener un dumpeado, utilizaremos el mismo plugin de Olly llamado Ollydump que va fenomenalmente bien al menos para mi gusto claro, las opciones las de siempre ;-)



Con estas opciones obtenemos un dumpeado que llamaremos tute.exe para variar un poco, pero si lo intentamos ejecutar...



típico ACCESS_VIOLATION (violación del espacio de memoria) esto lo indica el código asociado 0xc0000005 podríamos cargar el .exe en Olly y debuggearlo hasta dar con el error pero no hace falta ya que conocemos la causa: La IAT! recordemos que hemos dumpeado con la opción "Rebuild Impor" (reconstruir las importtaciones) desactivada pero bueno esto tiene remedio ;-)

Usando Import Reconstructor...

Pues eso mismo señores, vamos a ello, para los que no lo hayan utilizado antes les he de decir que no tengan ningún miedo que es muy fácil, recuerdo que la primera vez que yo lo utilicé fue con un tElock 0.91 y no me fue nada mal pues recuperó las importaciones como la nada.

En un primer momento cuando corres la aplicación puede parecer algo compleja por el mogollón de controles que tiene pero a medida que los leemos nos damos cuenta que es muy intuitiva de usar, que mejor que practicar con este UPX para aquellos que recién comiencen en este arte ;-)

Abrimos entonces la app y nos encontramos con la interfaz, en la parte superior encontramos un menú desplegable donde dice "Attach to an active process" y desde aquí seleccionamos la aplicación con la que trabajamos, debemos tener el programa original

(protegido con UPX) corriendo para que nos lo muestre entre los procesos activos, y una vez seleccionado ya podemos trabajar con él.

El segundo paso será suministrarle los valores correctos del OEP y IAT que son los que dice OEP, RVA y Size respectivamente todos ellos trabajan en offsets, es decir, si el OEP está en :401240 en el campo OEP pondríamos 1240h (401240-400000) imaginando que tenemos un imagebase igual a 400000, por último el campo Size especifica el tamaño de la propia IAT, fácilmente obtenible por cierto en la mayoría de programas, en este caso la hallaremos manualmente por si las moscas alguno no sabe bien como localizarla:

Primeramente buscamos un salto o llamada a la IAT, fácilmente reconocible, p.ej:

Address	Hex dump	Disassembly	Comment
004067E0	- FF25 70E24D00	JMP DWORD PTR DS:[4DE270]	kernel32.GetModuleHandleA
004067E6	8BC0	MOV EAX,EAX	
004067E8	- FF25 6CE24D00	JMP DWORD PTR DS:[4DE26C]	kernel32.LocalAlloc
004067EE	8BC0	MOV EAX,EAX	
004067F0	- FF25 68E24D00	JMP DWORD PTR DS:[4DE268]	kernel32.TlsGetValue
004067F6	8BC0	MOV EAX,EAX	
004067F8	- FF25 64E24D00	JMP DWORD PTR DS:[4DE264]	kernel32.TlsSetValue
004067FE	8BC0	MOV EAX,EAX	

ESTE!!

Ahí encontramos uno en :4067E0 y va a GetModuleHandleA, si pulsamos con el botón derecho sobre la instrucción y elegimos "Follow in dump -> Memory Address" caeremos en plena IAT, justo aquí:

Address	Hex dump	ASCII
004DE270	86 AD E5 77 A9 AD E5 77 00 00 00 00 B1 63 DB 77	
004DE280	10 24 DA 77 F4 6C DB 77 9A 22 DA 77 6D 69 DB 77	
004DE290	8B 6F DB 77 D4 65 DB 77 E2 68 DB 77 D6 27 DA 77	
004DE2A0	D8 17 DA 77 00 00 00 00 0A 4E E5 77 3A F1 E5 77	
004DE2B0	35 FD E4 77 12 AC E5 77 44 7F E5 77 72 AC E5 77	
004DE2C0	EA 1B E4 77 CF D2 E4 77 35 3C E9 77 5D 26 E5 77	
004DE2D0	A6 9D E5 77 4B 2F E5 77 2E F0 E5 77 07 2E E5 77	

Vemos que el salto seguido apunta a [4DE270] donde está la dirección 77E5AD86 qe es la de GetModuleHAndleA en mi máquina, si desde aquí subimos haciendo un scroll encontraremos el inicio de tabla fácilmente:

Address	Hex dump	ASCII
004DE148	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004DE158	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004DE168	DF A7 E5 77 CA 25 F4 77 90 56 F6 77 DE 55 F6 77	
004DE178	45 A7 E5 77 CB 15 E6 77 72 AC E5 77 A0 60 E5 77	
004DE188	82 A6 E5 77 71 A6 E5 77 60 A6 E5 77 44 7F E5 77	
004DE198	49 A9 E5 77 FC A7 E5 77 E1 60 E5 77 4E 27 E5 77	
004DE1A8	0A 4E E5 77 41 D9 E5 77 41 DA E5 77 7E 17 E4 77	
004DE1B8	32 B3 E5 77 86 AD E5 77 A9 AD E5 77 62 73 E5 77	
004DE1C8	02 15 F4 77 58 E3 E5 77 8C E6 E5 77 0D A1 E5 77	

Vemos como los valores pasan de ser del tipo :77xxxxxx a 00000000, esta brecha delimita la IAT perfectamente y encontramos su inicio, lo anotamos:

inicio_iat: 4DE168

Ahora hacemos lo mismo pero para abajo buscando su fin:

Address	Hex dump	ASCII
004DE868	25 74 32 77 A4 7F 33 77 57 A4 31 77 3D 51 31 77	
004DE878	E3 AD 31 77 B1 38 31 77 00 00 00 00 B9 29 40 77	
004DE888	9B 2B 47 77 00 00 00 00 7C D5 3C 77 31 27 40 77	
004DE898	56 B0 3F 77 C2 35 4B 77 00 00 00 00 CF 90 36 76	
004DE8A8	90 23 36 76 00 00 00 00 F6 10 0E 75 26 16 0E 75	
004DE8B8	CA 59 0E 75 71 10 0E 75 00 00 00 00 A5 A1 B0 76	
004DE8C8	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004DE8D8	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004DE8E8	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Lo encontramos de la misma manera, es cuestión de anotar el dato:

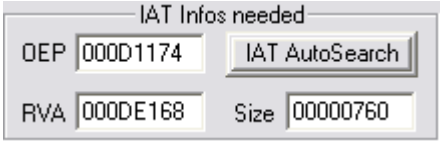
final_iat: 4DE8C8

le damos un DWORD de margen por si acaso, pues no está de más hacerlo, realmente acaba en 4DE8C4, esto no importa, calculemos su largo:

largo=final-inicio => 4DE8C8-4DE168=760

largo_iat: 760

Vale pues ya tenemos todos los datos necesarios para el ImpRec, y ya es sólo cuestión de rellenar los campos correctamente, de esta manera:



Allí están los valores obtenidos, llegó el momento de reparar el dumpeado, pulsamos "Fix dump" y elejimos tute.exe, al poco nos informará que el archivo se ha reparado y ahora se llamará tute_.exe (le añade el guión final) este por su puesto tiene la IAT reparada y si lo ejecutamos ya funciona ;-DDD

Superando las limitaciones...

Pues bien tras todo esto sólo nos queda evitar las limitaciones y/o restricciones que el programa pueda tener, hasta ahora nos hemos limitado a descomprimir el ejecutable nada más, veamos qué protección tiene, al ejecutarlo por primera vez veíamos la pantallita de configuración, si pulsábamos en el botón "Register" se abría el cuadro de registro donde ingresábamos nuestros datos, al fallar mostraba este mensaje:

Pues vamos a atacarlo, jeje.. la API que nos puede ayudar es EnableWindow, le ponemos un BP y traceamos a lo retro cuando salte, para ello deberemos poner un serial y aceptar, en breve Olly para la ejcución, traceamos un poco para llegar a un bucle en el que deja el mensaje flotando en pantalla:

Address	Hex dump	Disassembly	Comment
00449F88	50	PUSH EAX	
00449F89	E8 9ED3FBFF	CALL <JMP.&user32.SendMessageA>	
00449F8E	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00449F91	33D2	XOR EDX,EDX	
00449F93	8990 34020000	MOV DWORD PTR DS:[EAX+234],EDX	
00449F99	8B03	MOV EAX,DWORD PTR DS:[EBX]	;inicio_loop_MsgBox
00449F9B	E8 D42D0000	CALL tute_.0044CD74	
00449FA0	8B03	MOV EAX,DWORD PTR DS:[EBX]	
00449FA2	80B8 8C000000	CMP BYTE PTR DS:[EAX+8C],0	
00449FA9	74 0F	JE SHORT tute_.00449FBA	
00449FAB	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00449FAE	C780 34020000	MOV DWORD PTR DS:[EAX+234],2	
00449FB8	EB 14	JMP SHORT tute_.00449FCE	
00449FBA	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00449FBD	83B8 34020000	CMP DWORD PTR DS:[EAX+234],0	
00449FC4	74 08	JE SHORT tute_.00449FCE	
00449FC6	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00449FC9	E8 26FDFFFF	CALL tute_.00449CF4	
00449FCE	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
00449FD1	8B80 34020000	MOV EAX,DWORD PTR DS:[EAX+234]	
00449FD7	85C0	TEST EAX,EAX	
00449FD9	74 BE	JE SHORT tute_.00449F99	;fin_loop!!
00449FDB	8945 F8	MOV DWORD PTR SS:[EBP-8],EAX	
00449FDE	6A 00	PUSH 0	

Ahí queda el mensaje, al cerrarlo paramos en el BP que tenemos puesto y que vemos en la imagen, pero esto no es todo, si seguimos traceando a lo retro (llegando hasta el próximo RETN y pasándolo cada vez) nos encontraremos con una grata sorpresa y es la zona caliente de la comprobación del serial, es esta:

```
Address   Hex dump   Disassembly                          Comment
004B2E88  .  E8 3710F5FF  CALL tute_.00403EC4
004B2E8D  .  83F8 23      CMP EAX,23                      hmmm...
004B2E90  .v 74 13        JE SHORT tute_.004B2EA5         ;buen salto!
004B2E92  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
004B2E97  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2E99  .  BA 30304B00  MOV EDX,tute_.004B3030          ASCII "Wrong License Key !"
004B2E9E  .  E8 513FFFFF  CALL tute_.004A6DF4
004B2EA3  .v EB 6F        JMP SHORT tute_.004B2F14
004B2EA5  >  807D F3 00   CMP BYTE PTR SS:[EBP-D],0
004B2EA9  .v 74 58        JE SHORT tute_.004B2F03
004B2EAB  .  BA C02F4B00  MOV EDX,tute_.004B2FC0          ASCII "Thank you for using BG CD Manag
004B2EB0  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
004B2EB5  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2EB7  .  E8 383FFFFF  CALL tute_.004A6DF4
004B2EBC  .  6A 00        PUSH 0                          ShowState = SW_HIDE
004B2EBE  .  A1 98984D00  MOV EAX,DWORD PTR DS:[4D9898]
004B2EC3  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2EC5  .  8B40 24      MOV EAX,DWORD PTR DS:[EAX+24]
004B2EC8  .  50           PUSH EAX                        hWnd
004B2EC9  .  E8 3645F5FF  CALL <JMP.&user32.ShowWindow>   ShowWindow
004B2ECE  .  6A 05        PUSH 5
004B2ED0  .  8D55 BC      LEA EDX,DWORD PTR SS:[EBP-44]
004B2ED3  .  33C0         XOR EAX,EAX
004B2ED5  .  E8 5EFAF4FF  CALL tute_.00402938
004B2EDA  .  8D45 BC      LEA EAX,DWORD PTR SS:[EBP-44]
004B2EDD  .  BA 08304B00  MOV EDX,tute_.004B3008          ASCII " -nomutex"
004B2EE2  .  E8 E50FF5FF  CALL tute_.00403ECC
004B2EE7  .  8B45 BC      MOV EAX,DWORD PTR SS:[EBP-44]
004B2EEA  .  E8 9911F5FF  CALL tute_.00404088
004B2EEF  .  50           PUSH EAX                        CmdLine
004B2EF0  .  E8 573DF5FF  CALL <JMP.&kernel32.WinExec>    WinExec
004B2EF5  .  A1 98984D00  MOV EAX,DWORD PTR DS:[4D9898]
004B2EFA  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2EFC  .  E8 BBA0F9FF  CALL tute_.0044CFBC
004B2F01  .v EB 11        JMP SHORT tute_.004B2F14
004B2F03  >  BA 30304B00  MOV EDX,tute_.004B3030          ASCII "Wrong License Key !"
004B2F08  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
004B2F0D  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2F0F  .  E8 E03EFFFF  CALL tute_.004A6DF4
```

En la parte superior vemos que compara EAX con 23h (35 decimal) quizá es el número de dígitos del serial que debe encontrar, para saberlo con certeza debemos poner un BP en el CALL y esperar que pare, allí veremos cuanto vale EAX y el número de dígitos que hayamos puesto, si son iguales es cierta nuestra suposición, en caso contrario pues nada, de momento pongamos el BP:

```
Address   Hex dump   Disassembly                          Comment
004B2E88  .  E8 3710F5FF  CALL tute_.00403EC4
004B2E8D  .  83F8 23      CMP EAX,23                      hmmm...
004B2E90  .v 74 13        JE SHORT tute_.004B2EA5         ;buen salto!
004B2E92  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
004B2E97  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2E99  .  BA 30304B00  MOV EDX,tute_.004B3030          ASCII "Wrong License Key !"
004B2E9E  .  E8 513FFFFF  CALL tute_.004A6DF4
004B2EA3  .v EB 6F        JMP SHORT tute_.004B2F14
004B2EA5  >  807D F3 00   CMP BYTE PTR SS:[EBP-D],0
004B2EA9  .v 74 58        JE SHORT tute_.004B2F03
004B2EAB  .  BA C02F4B00  MOV EDX,tute_.004B2FC0          ASCII "Thank you for using BG CD Manag
004B2EB0  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
```

Ahor introducimos un serial, por ejemplo este:

12121212454545457878787856565656

que como vemos tiene 32 dígitos 20h y al parar Olly en el BP tenemos:

```
Registers (MMX)
EAX 00000020
ECX 32313231
EDX 00000020
EBX 00AE29C8
ESP 0012F62C
```

jeje.. EAX==20h

Vaya pues nos vamos acercando ;-) ya sabemos que el serial debe contener 35 dígitos, al anterior le añadimos otros tres: 12121212454545457878787856565656123 y volvemos a aceptar, cuando Olly pare vemos que efectivamente EAX==23h vamos por la buena senda amigos, como la longitud es válida se produce el salto en :4B2E90

```
Address   Hex dump   Disassembly                          Comment
004B2E8D  .  83F8 23      CMP EAX,23                      hmmm...
004B2E90  .v 74 13        JE SHORT tute_.004B2EA5         ;buen salto!
004B2E92  .  A1 28984D00  MOV EAX,DWORD PTR DS:[4D9828]
004B2E97  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004B2E99  .  BA 30304B00  MOV EDX,tute_.004B3030          ASCII "Wrong License Key !"
004B2E9E  .  E8 513FFFFF  CALL tute_.004A6DF4
004B2EA3  .v EB 6F        JMP SHORT tute_.004B2F14
004B2EA5  >  807D F3 00   CMP BYTE PTR SS:[EBP-D],0
004B2EA9  .v 74 58        JE SHORT tute_.004B2F03
004B2EAB  .  BA C02F4B00  MOV EDX,tute_.004B2FC0          ASCII "Thank you for using BG CD Manag
```

Evitando el error en :4B2E9E pero aún no hemos terminado pues queda la comprobación de [EBP-D] y que debe valer 0 y no saltar después en :4B2EA9 y llegar así al mensaje de

felicitación, aunque de momento no es posible, hacemos un "Follow in dump -> Memory Address" y encontramos la variable de la que depende el registro:

```
Address   Hex dump                                          ASCII
0012F67B  00 78 A6 B0 00 28 21 B1 00 30 CC AE 00 F0 F7 12  .x▒▒.(!▒.0╠«.-→↕
0012F68B  00 84 FE 42 00 5C 59 47 00 9B 5B 47 00 90 88 AE  .ä■B.\YG.¢[G.ÉêÇ
0012F69B  00 70 5B 47 00 42 5B 47 00 B8 F8 12 00 34 F8 12  .p[G.B[G.®°↕.4°↕
0012F6AB  00 90 88 AE 00 F0 FC 42 00 B8 F8 12 00 34 F8 12  .ÉêÇ.-ⁿB.®°↕.4°↕
0012F6BB  00 90 88 AE 00 3B 28 43 00 B8 F8 12 00 90 88 AE  .ÉêÇ.;(C.®°↕.ÉêÇ
```

Estamos hablando lógicamente de [12F67B] que pertenece al espacio de la pila (stack) y no es ninguna variable del programa ya que no están en su espacio de memoria como sí sería por ejemplo [40F400] lo próximo que deberemos hacer es ver quién escribe en ella y porqué mete un 0 en lugar de otro valor, pero increíblemente si miramos por encima de este código encontramos esto:

```
Address   Hex dump     Disassembly                  Comment
004B2CB8  .- E9 D706F5FF  JMP tute_.00403394
004B2CBD  .  C645 F3 00   MOV BYTE PTR SS:[EBP-D],0
004B2CC1  .  E8 2A0AF5FF  CALL tute_.004036F0
004B2CC6  >  8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
004B2CC9  .  BA 682F4B00  MOV EDX,tute_.004B2F68      ASCII "4582-C616C69656C61-5722-010105-3521"
004B2CCE  .  E8 0113F5FF  CALL tute_.00403FD4
004B2CD3  .v 0F85 C800000 JNZ tute_.004B2DA1
004B2CD9  .  33C0         XOR EAX,EAX
004B2CDB  .  55           PUSH EBP
```

parece un serial, como se ve en la imagen puse un BP para ver qué hace, justo en el momento de parar sobre él si miramos en los registros vemos:

```
Registers (MMX)
EAX 00AECC30 ASCII "1212121245454545787878785656565 6123"
ECX 0012F688
EDX 004B2F68 ASCII "4582-C616C69656C61-5722-010105-3521"
EBX 00037761
ESP 0012F62C
```

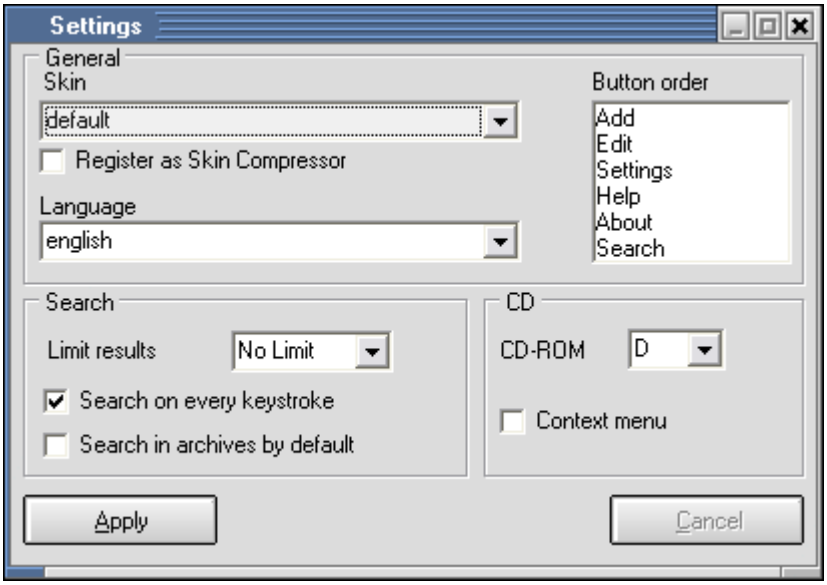
jooooder!! que es esto? parece que va a compararlo con el que nosotros pusimos, no puede ser, parecía que iba a ser más complicado ¿? bueno de momento lo probamos a ver:

Acepto y...

No puede ser... un hardcoded serial ¿? nos dice que la aplicación se debe reiniciar, jeje.. quizá es una absurda trampa para hacernos caer y luego ni se registra ni nada, bueno reiniciiemos de momento a ver si surge efecto...

Cerramos la aplicación, bueno en realidad se cierra sola y la volvemos a abrir para mi sorpresa de ver cómo realmente ha quedado registrado ¡¡!!

Veamos que aya no aparece el botón "Register" que antes aparecía en la parte inferior central de la pantalla principal, este es su renovado aspecto ;-)



Efectivamente quedó registrado, jeje.. una aplicación más o menos según se mire ;-)

En fin, que el único serial válido parece ser este:

4582-C616C69656C61-5722-010105-3521

poiniéndolo en la ventana de registro nos convertimos en nuevo usuario, la protección s francamente lamentable, sinceramente pensé quenos divertiríamos un poquitín más, de todos modos programar algo así no debe ser nada sencillo, y ya puestos podrían haberle puesto una protección más seria creo yo, un hardcoded serial no se ve por ahí muy a menudo, pero en fin si quieren que le crackeen su aplicación en 5 minutos allá ellos, nosotros por su puesto encantados ;-)

Y sin más que decir me despido sólo por el momento eh! espero volver al ataque con algo más serio pero bueno a veces se tercian así las cosas que le vamos a hacer, también he e decir que estoy escribiendo el tute sin saber la solución así que estas cosas pasan..

Para cualquier consulta, sugerencia o duda (lo que sea) me pueden contactar aquí:

syxe00@yahoo.es

Un abrazo para todo CracksLatinoS y FELIZ 2006..!!

SyXe'05